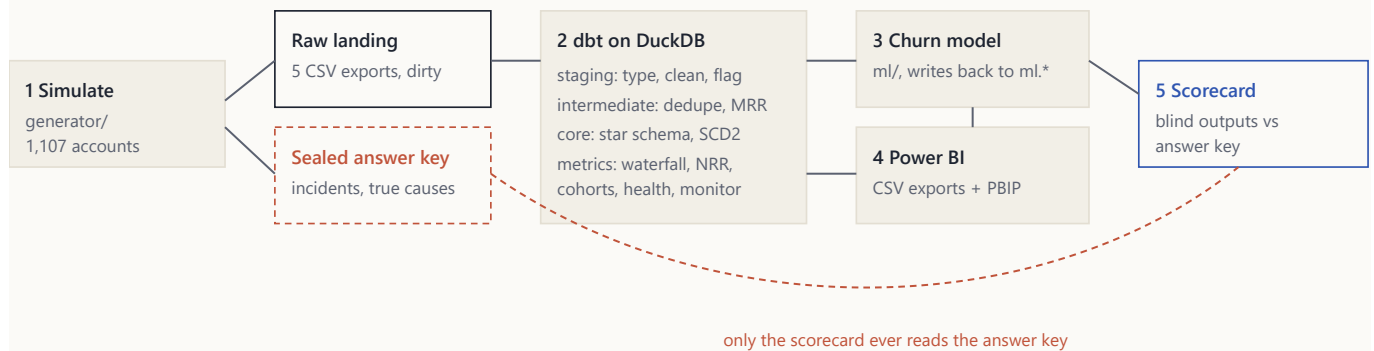


How it works

A stage-by-stage walk through the code: what each step does, where it lives, how it is proved, and the design choices most worth challenging.

SaaS Revenue Leak · Python, DuckDB, dbt, scikit-learn, XGBoost, Power BI · read with the [case study](#).

1. The flow in one picture



Checks at every stage: 105 dbt nodes (tests and enforced contracts) · 14 of 14 deliberate breakages caught by the mutation test

69 report numbers matched against DuckDB in Power BI's own engine · every text line measured to fit · 90 output files byte-identical on rerun

2. Running it

```

pip install -r requirements.txt
python pipeline.py                # everything, about 45 seconds
python scripts/mutation_test.py  # 14 deliberate breaks, each must fail a test
python scorecard/multi_seed.py --label v2 --seeds 7 2024 31337
  
```

The last command reruns the whole pipeline on simulated worlds that were never inspected, and writes a results table to `docs/`.

Then open `powerbi/RevenueLeak.pbip` in Power BI Desktop and press Refresh. The report reads the CSVs in `exports/` through the `DataFolder` parameter, which `build_pbip.py` sets to this checkout's path.

To prove the report: `powerbi/open_pbip.ps1 -Keep` opens it in its own Desktop window and refreshes it; `powerbi/check_report.py` then evaluates every measure and compares the report's numbers with SQL; `powerbi/check_fit.py` measures every line of text against the space it has.

3. Stage by stage

Simulate (`generator/`)

A day-by-day simulation in numpy. Every account carries a hidden health value between 0 and 1 that drives product sessions, support tickets, CSAT, seat expansion, downgrades and churn at renewal. `config.py` holds every parameter, including the five planted incidents (a Starter price rise, a broken release, a neglected account-manager book, duplicated Stripe webhooks and a weak partner cohort). When an account churns, the simulation writes down the true cause. `dirty_data.py` then adds 13 kinds of real-world mess (exact and near duplicates, missing values, mixed time zones, casing, impossible values, CRM changes recorded weeks late) and counts each one.

Staging (`dbt/models/staging`)

Raw CSVs are read as text and typed here. Exact re-deliveries are dropped, time zones normalised to UTC, casing fixed. Known mess is *flagged* rather than hidden: `warn` -level tests fire on every injected problem, so the build passes but the warning is the alert.

Intermediate (`dbt/models/intermediate`)

- `int_stripe_customer_accounts` maps Stripe customers to CRM accounts, falling back to the billing email domain when the account id is missing.
- `int_subscription_events` removes webhook duplicates that arrive under a new event id (same subscription, type, time, seats and MRR).
- `int_account_mrr_changes` and `int_account_mrr_daily` turn events into end-of-day MRR state and a dense daily revenue table.
- `int_customer_versions` builds SCD2 change points from CRM changes (by effective date, not recorded date) and from ARR band changes derived from billing.

Star schema (`dbt/models/marts/core`)

TABLE	GRAIN	NOTES
<code>fact_mrr_movements</code>	account x day of change	New, Expansion, Contraction, Churn, Reactivation; summing the deltas gives MRR at any date
<code>fact_product_usage</code> , <code>_weekly</code>	account x day, account x week	weekly active users are distinct users, not summed days
<code>fact_support_tickets</code>	ticket	first-response and resolution SLAs from a seed; invalid values nulled and flagged
<code>dim_customers</code>	account version	SCD Type 2 on account manager, CS tier, company size and ARR band; row -1 is the unknown member
<code>dim_date</code>	day	2024-2027 with a February fiscal year

Every core model has an enforced dbt contract, so a change in a column's name or type fails the build before Power BI ever sees it.

Metrics (`dbt/models/marts/metrics`)

- `mrr_waterfall_monthly` : month-end MRR per account against the previous month-end, classified. Churn is credited to the owner at the start of the month.
- `revenue_retention_monthly` : the company MRR bridge with NRR and GRR month over month and trailing 12 months. A singular test checks that the bridge balances every month.
- `cohort_retention` : logo and revenue retention by signup cohort and months since signup.
- `customer_health_weekly` : a rules-based score from usage trend, adoption, support load and CSAT, with weights fixed up front.
- `anomaly_alerts` : the revenue leak monitor, six detectors that each test a metric against its own trailing baseline with no look-ahead, and a `detected_at` date that only counts complete periods.

Churn model (`ml/churn_model.py`)

One feature row per paying account per week, built only from data up to that week: 30-day usage velocity, adoption, days since last session, ticket spike against the account's own baseline, high-priority and bug tickets, SLA breaches, CSAT level and trend, seat change, recent price rise, downgrades, days to renewal, plus segment, plan, channel and account manager. The label is "churns in the next 60 days". Training uses weeks up to Dec 2025; testing uses Mar to Aug 2026, out of time and through a release failure the model never saw. Logistic regression and XGBoost are both trained; whichever has the better precision-recall AUC on the test period is refitted and used. Each current account gets a probability, a risk tier and its top three drivers in plain words.

Power BI (`powerbi/build_pbip.py`)

The whole report is generated: a TMDL semantic model (13 tables, 12 relationships, 66 measures) and a PBIR report of four pages. Column types are read from the same DuckDB queries that write the CSVs, so the model and the data cannot drift. Big money values go through display measures that do their own scaling, so cards read the same in every Windows locale.

Scorecard (`scorecard/`)

`score.py` is the only code that opens the answer key. It decides whether each incident was found (rules written before the first run), lists every alert that matches no incident as a false alarm (unless most of the churn behind it was incident-driven, which makes it a secondary signal), compares each injected data problem with what the tests flagged, and measures how many churns the model and the health score flagged beforehand, and how early.

4. Design choices worth challenging

SCD2 from a change log, not dbt snapshots

Snapshots record what the warehouse saw at each run; the CRM here exports its own change history with effective dates. Building versions from effective dates puts late-recorded changes in the right place. The cost: history is only as good as the CRM's log.

A monthly waterfall nets changes inside a month

An account that adds seats and then cuts them in the same month shows as one net movement. That is the standard board view; the daily ledger keeps every change for anyone who needs it.

Health score weights were fixed, not fitted

The rules-based score is meant to be explainable to a CS team. The scorecard shows it catches far less churn than the model, and the case study says so; fitting its weights would just recreate the model.

The detectors were revised once after seeing results

That makes seed 42 unfit to judge them, so both versions were rerun on three unseen worlds. See [detector revisions.md](#).

Logistic regression beat XGBoost

Selection is by out-of-time PR AUC, not cross-validation on shuffled rows, because churn behaviour drifts. On every seed tried here the simpler model generalised better; the code still picks per run rather than assuming it.

DuckDB pinned to one thread

Parallel sums of doubles can differ in the last digit between runs. Every output of this project is byte-identical on rerun; on data this size the cost is a few seconds.

CSV into Power BI, not Parquet or a live database

CSV through Power Query is the path proven to open in Desktop without drivers; the CSVs are exported in a fixed order so diffs stay meaningful.

Synthetic data

The answer key exists only because the data is simulated. The results say how well the analytics find what was planted, not how a real company behaves.

5. What to review

IF YOU WANT TO SEE	OPEN
How incidents and mess are planted	<code>generator/config.py</code> , <code>generator/dirty_data.py</code>
Webhook dedupe and MRR state	<code>int_subscription_events.sql</code> , <code>int_account_mrr_changes.sql</code>
SCD2	<code>int_customer_versions.sql</code> , <code>dim_customers.sql</code> , <code>macros/generic_tests.sql</code>
Waterfall, NRR, GRR	<code>mrr_waterfall_monthly.sql</code> , <code>revenue_retention_monthly.sql</code>
The monitor	<code>anomaly_alerts.sql</code>
Leakage control in the model	<code>FEATURE_SQL</code> and the split constants in <code>ml/churn_model.py</code>
How results are judged	<code>scorecard/score.py</code> , <code>docs/scorecard.md</code>
Report as code and its checks	<code>powerbi/build_pbip.py</code> , <code>check_report.py</code> , <code>check_fit.py</code>

6. Known limits

- One business, simulated. Incident dates are the same in every seed; only the random world around them changes.
- The account-manager detector needs about six months of book history before its test has the power to fire, so a neglected book is found late.
- The health score rarely reaches At Risk; the model is the stronger early-warning signal by a wide margin.
- Power BI has no free public link, so the report ships as a project to open in Desktop, with images in the docs.

Every number in this guide comes from a pipeline run; reruns are byte-identical.